

飞机管道铺设研究课设报告

华中科技大学，人工智能与自动化学院，湖北武汉 430074

摘要—本文描述了一个飞机管道铺设研究项目，该项目旨在一个固定空间内，根据多个已知的起始点和目标点，在满足相应约束条件（如不能触碰障碍物等）下，设计相应的路线进行连接，以最小化总体路线长度或者管道铺设成本。

Index Terms—A* 算法；管道铺设的路径优化

I. 引言

在现代工业中，管道铺设是一个重要的环节，尤其是在飞机制造等领域。有效的管道铺设可以显著降低成本和提高安全性。本项目旨在通过计算机模拟和优化算法，解决管道铺设中的路径优化问题。

II. 问题描述

在固定空间内，需要根据多个已知的起始点和目标点，在满足不触碰障碍物等约束条件下，设计相应的路线进行连接，以最小化总体路线长度或者管道铺设成本。设计目标包括最小化管道弯折、最小化管道固定材料消耗和最小化管道交叉覆盖。

III. 解决方案

A. A* 算法的简介

本项目采用启发式智能算法和深度强化学习方法来解决路径规划问题。通过模拟网格环境，使用 A* 算法进行路径搜索，并考虑环境转换、奖励生成、状态更新等要素。

A* (A-Star) 算法是一种静态路网中求解最短路径最有效的直接搜索方法，也是解决许多搜索问题的有效算法，广泛应用于室内机器人路径搜索、游戏动画路径搜索等。它是图搜索算法的一种。

A* 算法是一种启发式的搜索算法，它是基于深度优先算法 (Depth First Search, DFS) 和广度优先算法 (Breadth First Search, BFS) 的一种融合算法，按照一定原则确定如何选取下一个结点。

启发式搜索算法指的是，从起点出发，先寻找起点相邻的栅格，判断它是否是最好的位置，基于这个最好的栅格再往外向其相邻的栅格扩展，找到一个此时最好的位置，通过这样一步一步逼近目标点，减少盲目的搜索，提高了可行性和搜索效率。

深度优先搜索算法的思想是，搜索算法从起点开始进行搜索 (初始状态下待搜索区域内所有结点均未被访问)，与周围所有邻点进行比较，选择其中距离终点最近的点进行存储，然后再以该邻点为基础对比其周围未被访问的所有邻点，仍然选择距离终点最近的邻点存储。若访问结点到达终点或访问完所有结点仍未到达终点，则视为搜索失败。成功搜索所存储的结点连接而成的路径即为起点到终点的路径。

广度优先搜索的原理是，从初始点出发依次访问与它相连接的邻点，访问完毕后再从这些邻点出发访问邻点的邻点，但是要保证先被访问的邻点的邻点要比后被访问的邻点的邻点先访问，直至图中所有已被访问的结点的邻点都被访问到。如果此时图中尚有未被访问的结点，则需要选取一个尚未被访问的结点作为新的初始点，继续搜索访问，直到图中所有的结点都被访问一遍为止。

因此深度优先算法与广度优先搜索算法从过程上存在较大差异。深度优先算法优先选择离目标点最近的结点，而广度优先搜索算法优先选择离初始点最近的点。基于深度优先算法，能以最快的速度找到一条连接初始点到目标点的路径，但不能保证路径的最优性 (例如以路径最短为标准)；广度优先搜索算法则必然能找到最短的路径，但由于需要遍历所有的结点，其计算复杂度更大。基于这两种算法的优缺点，A* 算法基于启发函数构建了代价函数，既考虑了新结点距离初始点的代价，又考虑了新结点与目标点距离的代价。

B. A* 算法的路径优劣评价公式

A* 算法使用一个路径优劣评价公式为：

$$f(n)=g(n)+h(n)$$

$f(n)$ 是从初始状态经由状态 n 到目标状态的代价估计, $g(n)$ 是从初始状态到状态 n 的实际代价, $h(n)$ 是从状态 n 到目标状态的最佳路径的估计代价。

A* 算法需要维护两个状态表, 分别称为 openList 表和 closeList 表。openList 表由待考察的节点组成, closeList 表由已经考察过的节点组成。

IV. 算法精讲

A. 预处理

如图, 假设我们需要从 A 点到目标点, 这两点之间有一堵墙。

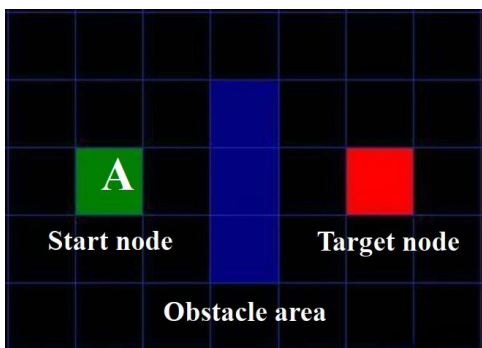


图 1.

首先, 我们把地图栅格化, 把每一个方格的中心称为节点; 这个特殊的方法把我们的搜索区域简化为了 2 维数组。数组的每一项代表一个格子, 它的状态就是可走 (walkable) 和不可走 (unwalkable)。通过计算出从 A 到目标点需要走过哪些方格, 就找到了路径。一旦路径找到了, 便从一个方格的中心移动到另一个方格的中心, 直至到达目的地。

B. 开始搜索

一旦我们把搜寻区域简化为一组可以量化的节点后, 我们下一步要做的便是查找最短路径。在 A* 中, 我们从起点开始, 检查其相邻的方格, 然后向四周扩展, 直至找到目标。从起点 A 开始, 定义 A 为父节点, 并把它加入到 openList 中。现在 openList 里只有起点 A, 后面会慢慢加入更多的项。如上图所示, 父节点 A 周围共有 8 个节点, 定义为子节点。将子节点中可达的 (reachable) 或者可走的 (walkable) 放入 openList 中, 成为待考察对象。若某个节点既未在 openList, 也没在 closeList 中, 则表明还未搜索到该节点。初始时, 节点

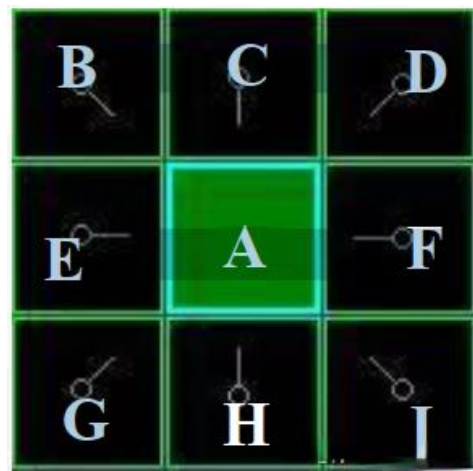


图 2.

A 离自身距离为 0, 路径完全确定, 将其移入 closeList 中; closeList 中的每个方格都是现在不需要再关注的。路径优劣判断依据是移动代价, 单步移动代价采取曼哈顿计算方式

$$d = |x_1 - x_2| + |y_1 - y_2|$$

即把横向和纵向移动一个节点的代价定义为 10。斜向移动代价为 14 (即 $\sqrt{2}$ 倍的横向或纵向移动代价)。现在 openList={B,C,D,E,F,G,H,I}, closeList = {A}。

下面我们需要去选择节点 A 相邻的子节点中移动代价 f 最小的节点, 下面以节点 I 的计算为例。移动代价评价函数为:

$$f(n)=g(n)+h(n)$$

$f(n)$ 是从初始状态经由状态 n 到目标状态的代价估计, $g(n)$ 是在状态空间中从初始状态到状态 n 的实际代价, $h(n)$ 是从状态 n 到目标状态的最佳路径的估计代价。

首先考察 g , 由于从 A 到该格子是斜向移动, 单步移动距离为 14, 故 $g=14$ 。再考察估计代价 h 。估计的含义是指忽略剩下的路径是否包含有障碍物 (不可走), 完全按照 Manhattan 计算方式, 计算只做横向或纵向移动的累积代价: 横向向右移动 3 步, 纵向向上移动 1 步, 总共 4 步, 故为 $h=40$ 。因此从 A 节点移动至 I 节点的总移动代价为: $f=g+h=54$ 。

以此类推, 分别计算当前 openList 中余下的 7 个子节点的移动代价 f , 从中挑选最小代价节点 F,

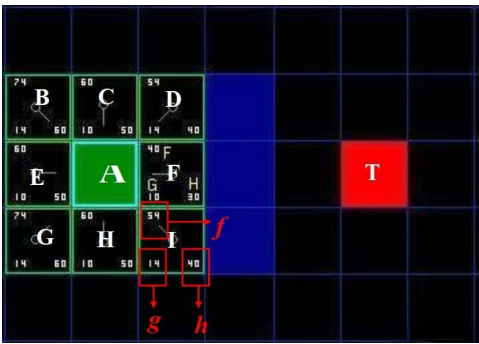


图 3.

移到 closeList 中。现在 $openList = \{B, C, D, E, G, H, I\}$, $closeList = \{A, F\}$ 。

C. 继续搜索

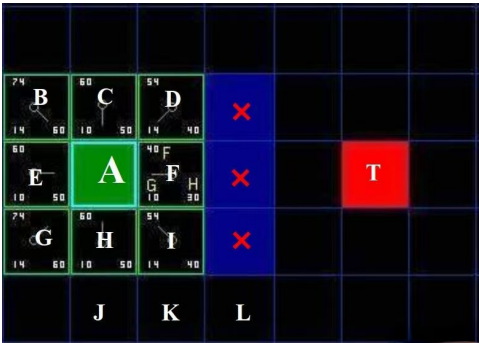


图 4.

从 openList 中选择 f 值最小的 (方格) 节点 I (D 节点的 f 值跟 I 相同, 任选其一即可, 不过从速度上考虑, 选择最后加入 openList 的方格更快。这导致了在寻路过程中, 当靠近目标时, 优先使用新找到的方格的偏好。对相同数据的不同对待, 只会导致两种版本的 A* 找到等长的不同路径), 从 openList 里取出, 放到 closeList 中。检查所有与它相邻的子节点, 忽略不可走 (unwalkable) 的节点、以及忽略已经存在于 closeList 的节点; 如果方格不在 openList 中, 则把它们加入到 openList 中, 并把它们作为节点 I 的子节点。如果某个相邻的节点 (假设为 X) 已经在 openList 中, 则检查这条路径是否更优, 也就是说经由当前节点 (我们选中的节点) 到达节点 X 是否具有更小的 g 值。如果没有, 不做任何操作。否则, 如果 g 值更小, 则把 X 的父节点设为当前方格, 然后重新计算 X 的 f 值和 g 值。

判断完所有子节点后, 现在 $openList = \{B, C, D, E, G, H, J, K, L\}$, $closeList = \{A, F, I\}$ 。依次类推,

不断重复。一旦搜索到目标节点 T, 完成路径搜索, 结束算法。

D. 确定实际路径

完成路径搜索后, 从终点开始, 向父节点移动, 这样就被带回到了起点, 这就是搜索后的路径。如下图所示。从起点 A 移动到终点 T 就是简单从路径上的一个方格的中心移动到另一个方格的中心, 直至目标。

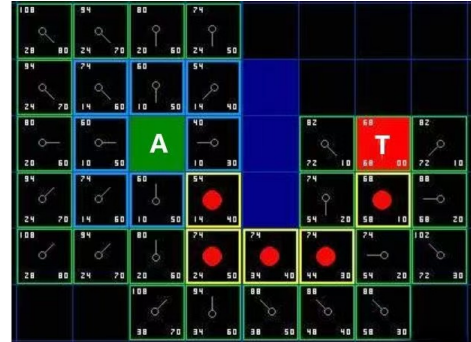


图 5.

V. 算法总结

A. 算法步骤

首先把起点加入 openList, 重复如下过程:

遍历 openList, 查找 f 值最小的节点, 把它作为当前要处理的节点。

把这个节点移到 closeList, 对当前方格的 8 个相邻方格:

1. 如果它是不可抵达的或者它在 closeList 中忽略它;
2. 如果它不在 openList 中, 则把它加入 openList, 并且把当前方格设置为它的父节点, 记录该方格的 f, g, h 值。
3. 如果它已经在 openList 中, 检查这条路径 (即经由当前方格到达它那里) 是否更好, 用 g 值作参考, 更小的 g 值表示这是更好的路径。如果 g 值更小, 把该节点的父节点设置为当前方格, 并重新计算它的 g 和 f 值。

停止搜索的情况有两种:

1. 把终点加入到了 openList 中, 此时路径已经找到了
2. 查找终点失败, 并且 openList 是空的, 此时没有路径。

保存路径。使用回溯的方法，从终点开始，每个方格沿着父节点移动直至起点，这就是最终搜索到的路径。

B. 伪代码

```

1 初始化open_list和close_list;
2 将起点加入open_list中，并设其移动代价为0;
3 如果open_list不为空，则从open_list中选取移动代价最小的节点n;
4   如果节点n为终点，则：
5       从终点开始逐步追踪parent节点，一直达到起点;
6       返回找到的结果路径，算法结束;
7   如果节点n不是终点，则：
8       将节点n从open_list中移除，并加入close_list中;
9       遍历节点n所有的邻近节点：
10          如果邻近节点m在close_list中，则：
11              跳过，选取下一个邻近节点
12          如果邻近节点m在open_list中，则：
13              计算起点经n到m的g值
14              如果此时计算出来的g值小于原来起点经m的原父节点到m的g值：
15                  更新m的父节点为n，重新计算m的移动代价f=g+h
16              否则，跳过
17          如果邻近节点m不在open_list也不在close_list中，则：
18              设置节点m的parent为节点n
19              计算节点m的移动代价f=g+h
20              将节点m加入open_list中

```

图 6.

VI. 代码实现

A. 运行思路

程序主要由三个部分组成：网格环境类'GridEnvironment'、A* 路径规划类'AStar' 和绘图函数'draw_paths'。

1) 网格环境类：

- 初始化：接收网格的宽度、高度和障碍物位置，初始化网格环境。
- is_valid：检查给定的坐标是否有效，即不在障碍物上且在网格范围内。
- random_free_space：生成指定数量的随机点，确保这些点不在障碍物上。

2) A* 路径规划类：

- 初始化：接收网格环境作为参数，初始化 A* 算法所需的数据结构。
- heuristic：使用曼哈顿距离作为启发式函数，估计从当前节点到目标节点的距离。
- search：执行 A* 搜索算法，找到从起点到终点的最短路径。
- reconstruct_path：根据 A* 算法中记录的父节点信息，重建从起点到终点的路径。

3) 绘图函数：

- draw_paths：绘制网格环境、障碍物、路径以及起点和终点，以直观展示路径规划结果。

B. 运行方式

- 1) 定义网格的宽度和高度，以及障碍物的位置。
- 2) 初始化 'GridEnvironment'和 'AStar'对象。
- 3) 生成随机的起点和终点。
- 4) 对每对起点和终点，使用 'AStar'对象搜索路径，并将结果存储在路径列表中。
- 5) 调用 'draw_paths'函数，绘制并显示路径规划结果。

C. 源代码

```

1 import heapq
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import random
5
6 # 定义网格环境
7 class GridEnvironment:
8     def __init__(self, width, height, obstacles):
9         self.width = width
10        self.height = height
11        self.obstacles = obstacles # 障碍物位置
12
13    def is_valid(self, x, y):
14        """判断节点是否有效"""
15        return 0 <= x < self.width and 0 <= y < self.height and (x, y) not in self.obstacles
16
17    def random_free_space(self, num_points):
18        """生成指定数量的随机点，确保它们不在障碍物上"""
19        free_points = set()
20        while len(free_points) < num_points:
21            x = random.randint(0, self.width - 1)
22            y = random.randint(0, self.height - 1)
23            if (x, y) not in self.obstacles:
24                free_points.add((x, y))
25        return list(free_points)
26
27    # A* 路径规划
28    class AStar:
29        def __init__(self, environment):
30            self.env = environment
31
32        def heuristic(self, x1, y1, x2, y2):
33            """曼哈顿距离作为启发式函数"""
34            return abs(x1 - x2) + abs(y1 - y2)
35

```

```

36 def search(self, start, goal):
37     """A* 搜索"""
38     open_set = []
39     heapq.heappush(open_set, (0 + self.heuristic(*
40         start, *goal), start)) # (优先级, 节点)
41     came_from = {}
42     g_score = {start: 0}
43     f_score = {start: self.heuristic(*start, *goal
44         )}
45     while open_set:
46         _, current = heapq.heappop(open_set)
47         if current == goal:
48             return self.reconstruct_path(came_from,
49                 current)
50     x, y = current
51     for dx, dy in [(0, 1), (1, 0), (0, -1), (-1,
52         0)]:
53         neighbor = (x + dx, y + dy)
54         if not self.env.is_valid(*neighbor):
55             continue
56         tentative_g_score = g_score[current] + 1
57         if neighbor not in g_score or
58             tentative_g_score < g_score[neighbor]:
59             came_from[neighbor] = current
60             g_score[neighbor] = tentative_g_score
61             f_score[neighbor] = tentative_g_score + self.
62                 heuristic(*neighbor, *goal)
63             heapq.heappush(open_set, (f_score[neighbor],
64                 neighbor))
65     return None
66 def reconstruct_path(self, came_from, current):
67     :
68     """重建路径"""
69     path = []
70     while current in came_from:
71         path.append(current)
72         current = came_from[current]
73     path.append(current)
74     return path[::-1]
75     # 绘制图像

76 def draw_paths(env, paths, starts, goals,
77     path_labels):
78     plt.figure(figsize=(15, 15))
79     grid = np.zeros((env.height, env.width))
80     # 绘制障碍物
81     for x, y in env.obstacles:
82         grid[y, x] = 1
83
84     plt.imshow(grid, cmap="Reds", origin="lower",
85         alpha=0.5) # 使用红色系颜色图表示障碍物
86     # 绘制路径
87     colors = ['blue', 'green', 'yellow', 'red', '
88         cyan', 'magenta', 'black', 'grey']
89     start_colors = ['orange', 'lightgreen', 'pink'
90         , 'gold', 'lightblue', 'violet', 'brown',
91         'tan']
92     goal_colors = ['darkorange', 'forestgreen', '
93         darkmagenta', 'darkred', 'navy', 'indigo',
94         'dimgray', 'sienna']
95     for i, (path, start, goal) in enumerate(zip(
96         paths, starts, goals)):
97         if path: # 确保路径不为空
98             x, y = zip(*path)
99             plt.plot(x, y, color=colors[i % len(colors)],
100                 linewidth=1, alpha=0.8, label=path_labels[
101                     i] if i < len(path_labels) else "")
102             plt.scatter(*start, color=start_colors[i % len(
103                 start_colors)], s=50, label=f"Start {i+1}"
104                 " if i == 0 else "")
105             plt.scatter(*goal, color=goal_colors[i % len(
106                 goal_colors)], s=50, label=f"Goal {i+1}"
107                 " if i == 0 else "")
108     plt.title("Pipe Planning Visualization")
109     plt.grid(True)
110     plt.legend(loc='center left', bbox_to_anchor
111         =(1, 0.5), title="Paths and Points")
112     plt.show()
113     # 主程序
114 def main():
115     # 定义网格和障碍物
116     width, height = 60, 60
117     # 障碍物分块在不同区域出现
118     obstacles = (
119         {(x, y) for x in range(10, 20) for y in range
120             (10, 20)} # 第一块障碍物区域

```

```

109 | {(x, y) for x in range(40, 50) for y in
      range(10, 20)} # 第二块障碍物区域
110 | {(x, y) for x in range(10, 20) for y in
      range(40, 50)} # 第三块障碍物区域
111 | {(x, y) for x in range(40, 50) for y in
      range(40, 50)} # 第四块障碍物区域
112 | {(x, y) for x in range(20, 40) for y in
      range(20, 40)} # 第五块障碍物区域
113 )
114
115 # 初始化环境
116 env = GridEnvironment(width, height, obstacles
      )
117 astar = AStar(env)
118
119 # 生成随机起点和终点
120 num_paths = 50 # 定义路径数量
121 starts = env.random_free_space(num_paths)
122 goals = env.random_free_space(num_paths)
123
124 paths = []
125 path_labels = [f"Path {i+1}" for i in range(
      num_paths)] # 创建路径标签
126 for start, goal in zip(starts, goals):
127     path = astar.search(start, goal)
128     if path:
129         paths.append(path)
130
131 # 绘制结果
132 draw_paths(env, paths, starts, goals,
      path_labels)
133
134 if __name__ == "__main__":
135     main()

```

D. 运行结果

VII. 总结

A. 结果

通过实验，我们发现所提出的算法能够有效地在复杂环境中找到最优路径，减少了管道弯折和交叉覆盖，同时降低了材料消耗。

B. A* 算法的意义

1. 高效路径寻找：A* 算法结合了最佳优先搜索和 Dijkstra 算法的优点，能够有效在复杂环境中找到最短

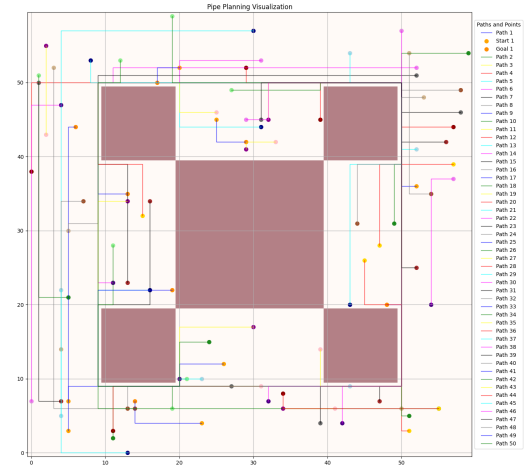


图 7.

路径。这使得其在机器人导航、游戏开发和地理信息系统（GIS）等领域得到广泛应用。

2. 适应性强：通过调整启发式函数，A* 算法可以灵活地适应不同的场景需求。可以在不同的环境中使用不同的启发式方法，从而优化搜索效率。

3. 可扩展性：实现中提供了良好的模块化结构，使代码容易扩展和修改。开发者可以轻松地对算法进行进一步的优化或添加额外的功能，例如支持三维空间的路径规划或动态环境下的实时路径调整。

4. 教育价值：该代码提供了一个清晰的实现范例，对于学习和理解图搜索算法的基本原理十分有帮助。学生和开发者可以基于此代码进行实验，深化对算法的理解。

5. 工业应用：在实际应用中，A* 算法能够应用于自动驾驶、无人机导航和移动机器人等多种场景，帮助这些系统高效、准确地完成复杂的导航任务。

参考文献

- [1] 武延涛, 张震, 刘伟良, and 卜予涵, "基于改进 A* 算法的全局路径规划"
- [2] 高九州 and 刘育航, "基于改进 A* 算法和再优化的避障路径规划"
- [3] 高九州 and 徐威峰, "基于改进 A 星算法的无人机自主避障路径规划"